

DATA 607 - Project 1: Chess Tournament Data Extraction

Muhammad Imran

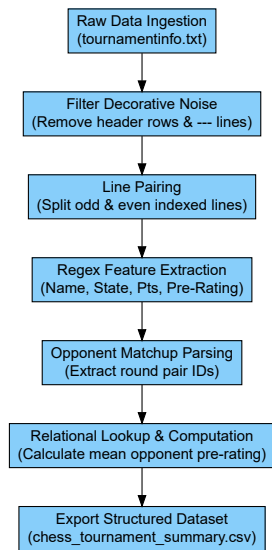
2026-09-23

Contents

Part 1: Comprehensive Project Approach	1
Detailed Step by Step Breakdown	3
Step 1: Data Ingestion and Sanitization	3
Step 2: Reconstructing Records via Dual-Vector Pairing	3
Step 3: Regular Expression (Regex) Extraction	3
Step 4: Extracting Opponents & Calculating Means	4
Step 5: Assembling and Exporting the Final Output	5

Part 1: Comprehensive Project Approach

In order to process unstructured or semi-structured text data into a structured format using R, one has to follow a systematic data processing pipeline. The following chart shows the pipeline’s execution flow.



Detailed Step by Step Breakdown

Step 1: Data Ingestion and Sanitization

When reading semi-structured text in R, `readLines()` ingests the document as a vector of character strings, where each element corresponds to one line of the text file.

```
raw_data <- readLines("tournamentinfo.txt")
```

The raw text file contains decorative border dividers (lines composed of dashes —) and header labels. We clean the vector using pattern matching:

```
# Remove border lines starting with dashes
cleaned_lines <- raw_data[!str_detect(raw_data, "^-+")]

# Remove the two column header rows at the top
cleaned_lines <- cleaned_lines[-c(1, 2)]
```

- `str_detect(raw_data, “^-+”)`: Finds all rows starting with one or more dashes. The `!` negates this condition to keep only content rows.
- likewise `[-c(1, 2)]`: Drops the first two rows, which contain column labels like `Pair | Player Name` and `USCF ID`.

Step 2: Reconstructing Records via Dual-Vector Pairing

Each player entry spans two consecutive rows

- Row 1: Player ID, Name, Total Points, and the 7 Round results.
- Row 2: State, USCF ID, Pre-Rating, and Post-Rating

To handle this cleanly without looping line-by-line, we separate the vector into odd and even elements:

```
line1 <- cleaned_lines[seq(1, length(cleaned_lines), by = 2)] # Rows 1, 3, 5...
line2 <- cleaned_lines[seq(2, length(cleaned_lines), by = 2)] # Rows 2, 4, 6...
```

By doing this, `line1[i]` and `line2[i]` always refer to player `i`.

Step 3: Regular Expression (Regex) Extraction

We extract key attributes from `line1` and `line2` using specific regex patterns:

1. Pair ID (from line1)

```
pair_id <- as.numeric(str_trim(str_extract(line1, "^\\s*\\d+")))
```

Matches leading spaces (`^\\s*`) followed by digits (`\\d+`) at the beginning of the string.

2. Player Name (from line1)

```
player_name <- str_trim(str_extract(line1, "(?<=\\|)[^|]+(?=\\|)"))
```

- Lookbehind assertion checking for a pipe (`|`) before the name.

- Matches one or more characters that are not a pipe.
- Lookahead assertion checking for a pipe (|) following the name.
- Removes leading and trailing padding spaces.

3. Player State (from line2)

```
state <- str_trim(str_extract(line2, "^\\s*[A-Z]{2}"))
```

Matches leading spaces followed by exactly two uppercase state letters (e.g., ON, MI, OH).

4. Total Points (from line1)

```
total_pts <- as.numeric(str_trim(str_extract(line1, "(?<=\\|)\\d+\\.\\d+")))
```

The argument, finds decimal point values (e.g., 6.0, 5.5) following a pipe delimiter.

5. Pre-Tournament Rating (from line2)

```
pre_rating <- as.numeric(str_extract(str_extract(line2, "R:\\s*\\d+"), "\\d+"))
```

- Edge Case Handling: Some ratings include provisional game counts (e.g., 1641P17 or 1220P13). * * *
First pass: `str_extract(line2, "R:\\s*\\d+")` grabs R: followed by the numeric digits, stopping before any P letter.
- Second pass: `str_extract(..., "\\d+")` strips out R: to leave only pure integer digits.

Step 4: Extracting Opponents & Calculating Means

1. Extracting Opponent Pair IDs

In line1, character position 48 onward contains the round-by-round results:

```
opponents_list <- str_extract_all(str_sub(line1, start = 48), "\\b\\d+\\b")
```

- `str_sub(line1, start = 48)`: Targets the 7 round result columns.
- Extracts word-bounded digits (e.g., in W 39, it extracts 39; in D 12, it extracts 12).
- Letters representing non-played rounds (e.g., H for half-point bye, U for unplayed, B for bye) are automatically ignored because they don't contain opponent pair IDs.

2. Vectorized Lookup & Mean Calculation

We define a helper function to calculate the mean opponent rating for a single player:

```
calc_avg_opp_rating <- function(opp_ids, rating_lookup) {
  ids <- as.numeric(opp_ids)
  ratings <- rating_lookup[ids]
  return(round(mean(ratings, na.rm = TRUE)))
}
```

How this lookup works step-by-step for Gary Hua (Player 1):

- Extracted Opponent IDs: `c(39, 21, 18, 14, 7, 12, 4)`.
- Lookup Ratings: `rating_lookup[c(39, 21, 18, 14, 7, 12, 4)]` maps directly to `c(1436, 1563, 1600, 1610, 1649, 1663, 1716)`.

c. Compute Mean:

$$\text{Mean} = \frac{1436 + 1563 + 1600 + 1610 + 1649 + 1663 + 1716}{7} = 1605.285\dots$$

d. Round Result:

We then apply this helper function across all players using `sapply()`:

```
avg_opp_rating <- sapply(opponents_list, calc_avg_opp_rating, rating_lookup = pre_rating)
```

Step 5: Assembling and Exporting the Final Output

Finally, we combine all extracted components into a tidy tibble dataframe and export it to a standard CSV file:

```
final_df <- tibble(  
  `Player's Name` = player_name,  
  `Player's State` = state,  
  `Total Number of Points` = total_pts,  
  `Player's Pre-Rating` = pre_rating,  
  `Average Pre Chess Rating of Opponents` = avg_opp_rating  
)  
  
# Save dataframe to disk  
write_csv(final_df, "chess_tournament_summary.csv")
```